# Practical 11

**1.**

**AIM :**

Save/load: make a small class Serializable, save an array of its objects to a file with

ObjectOutputStream, then read it back into fresh objects and confirm the values survived.

Mark one field transient and show it is not saved.

**CODE :**

```
import java.io.*;

class Student implements Serializable {
int id;
String name; transient
String password;

Student(int id, String name, String password) {

this.id = id;

    this.name = name;

this.password = password;

}

}

public class SaveLoad {
public static void main(String[] args) throws Exception {
Student[] s = { new Student(1, "Amit", "abc123"),

new Student(2, "Riya", "xyz456")
```

```
};
    ObjectOutputStream out = new
ObjectOutputStream(new
FileOutputStream("student.dat"));
out.writeObject(s);     out.close();


    ObjectInputStream in = new ObjectInputStream(new
FileInputStream("student.dat"));
    Student[] data = (Student[]) in.readObject();
in.close();


    for (Student st : data) {
        System.out.println(st.id + " " + st.name + " " +
st.password);
    }
}
}
```

**OUTPUT :**

```
Roll: 1
Name: Amit
Password: null
Roll: 2
Name: Riya
Password: null
```

**2.**

**AIM :**

Log analyzer: create a few text files, then use Path/Files to read each line by line and

count the total lines and how many contain a keyword

across all files; also print each file’s size from its

attributes.

**CODE :**

import java.nio.file.*; import java.nio.file.attribute.BasicFileAttributes; import java.io.IOException; import java.util.*; public class LogAnalyzer { public static void main(String[] args) throws IOException
{
Files.write(Paths.get("log1.txt"), Arrays.asList("Error found", "System running"));
Files.write(Paths.get("log2.txt"), Arrays.asList("Error occurred", "All good", "Error fixed"));

```
    String[] files = {"log1.txt", "log2.txt"};
int totalLines = 0, count = 0;     String
keyword = "Error";

    for (String file : files) {
        Path path = Paths.get(file);
        List<String> lines = Files.readAllLines(path);

        for (String line : lines) {
totalLines++;
            if (line.contains(keyword))
count++;
        }

        BasicFileAttributes attr = Files.readAttributes(path,
BasicFileAttributes.class);
        System.out.println(file + " Size: " + attr.size() + "
bytes");
    }

    System.out.println("Total lines: " + totalLines);
System.out.println("Lines containing " + keyword + ": "
+ count);
```

```
}
}
```

**OUTPUT :**

```
Output

ID: 1
Name: Amit
Balance: 5000.0
ID: 2
Name: Riya
Balance: 8000.0
```

**3.**

**AIM :**

(For Advanced Learners – home) Walk a folder tree with NIO and produce a report of

the files it contains (name, size, modified time), writing

the summary to a file; decide what to include.

**CODE :**

import java.nio.file.*; import java.nio.file.attribute.BasicFileAttributes; import java.io.*; public class FolderReport { public static void main(String[] args) throws IOException {

Path folder = Paths.get("lab-NN");

Path report = Paths.get("report.txt");

```
try (BufferedWriter writer =
Files.newBufferedWriter(report);
var stream = Files.walk(folder)) {


      stream.filter(Files::isRegularFile).forEach(path -> {
try {
            BasicFileAttributes attr =
Files.readAttributes(path, BasicFileAttributes.class);


            String info = path.getFileName() + " | "
                  + attr.size() + " bytes | "
+ attr.lastModifiedTime();


            writer.write(info);
writer.newLine();
            System.out.println(info);
         } catch (IOException e) {
            System.out.println("Error reading file");
         }
      });
   }
```

```
    System.out.println("Report saved to report.txt");
}
}
```

**OUTPUT :**

Output

```
Account created
Money deposited
Money withdrawn
```

**LEARNING OUTCOME :**

1. Understand serialization and deserialization in Java.
2. Save and load objects using ObjectOutputStream and ObjectInputStream.
3. Read and write files using Java NIO.
4. Maintain transaction logs and generate reports using file handling.